

Annex 6: How to Make Better Use of Secondary Development Functions

Instruction Manual

Instruction Manual Version: V2.05

Update Date: 2023.07.01

CONTENTS

1、 VCI_OpenDevice Function	1
2、 VCI_CloseDevice Function	1
3、 VCI_InitCAN Function	1
4、 VCI_GetReceiveNum Function	2
5、 VCI_StartCAN Function	2
6、 VCI_Receive Function	2
7、 VCI_Transmit Function	3
8、 VCI_FindUsbDevice and VCI_FindUsbDevice2 Function	5

1、 VCI_OpenDevice Function

This function is used to connect and open the USB-CAN adapter plugged into the computer.

In this function to open the USB-CAN adapter, where the computer process will automatically generate a handle for the USB-CAN operation, and at the same time in the memory to establish the relevant resources. This function must appear in pairs with VCI_CloseDevice. **After calling VCI_OpenDevice, when exiting the process, closing the software, reopening the adapter, etc., you must call VCI_CloseDevice function to release the resources, or else it may cause the process to crash, communication errors and other unknown errors!**

2、 VCI_CloseDevice Function

This function is used to close the opened USB-CAN adapter, after closing, the adapter will no longer send and receive activities until the next open start.

This function should appear in pairs with the VCI_OpenDevice function to close the adapter while releasing system resources!

3、 VCI_InitCAN Function

This function is called after a successful call to the VCI_OpenDevice function, and is used to initialize a CAN channel on the adapter. The formal parameter pInitConfig of this function passes the initialization-related parameters, including filter parameters, baud rate, and operating mode.

Attention:

1. When the CAN bus is sending and receiving data normally, try not to modify the CAN bus parameters or shut down the CAN bus through the USBCAN adapter, and wait until the data sending and receiving stops or detach the USBCAN adapter from the CAN bus before carrying out the corresponding operation.

4、 VCI_GetReceiveNum Function

This function must be called in the CAN open state, used to get in the CAN adapter a channel buffer has been received but not read by the VCI_Recive function of the number of frames.

In actual programming, memory is generally not tight, so generally do not have to call VCI_GetReceiveNum, in order to save CPU resources, and directly call VCI_Receive, receive with the pReceive structure array length can be set to 2500 frames pReceive [2500], each time to receive the maximum length of Len is also set to 2500, Len = 2500. If data is received, the function returns the corresponding number of frames and fills the pReceive structure, if there is no data, the function returns 0. So there is no need to call VCI_GetReceiveNum.

5、 VCI_StartCAN Function

This function is used in the VCI_InitCAN function called after the success of the function is to start a CAN channel, the start of the CAN channel must be initialized by the VCI_InitCAN function in order to start successfully, the channel is started, you can call the VCI_Receive and VCI_Transmit function to receive and send CAN messages.

6、 VCI_Receive Function

This function must be called after the CAN channel startup to read the CAN channel receive buffer (device driver to provide each channel 2500 frames of CAN message length of the receive buffer, which provides the application program to provide sufficient time to react to the processing time.) If the number of frames in the receive buffer is less than or equal to Len, all data will be read out; if the number of frames in the receive buffer is greater than Len, Len data will be read out. For the application program, if the data storage space of 2500 frames does not take up too much resources, Len can be set to 2500 and above, which ensures that all the data can be read back at once every time the VCI_Receive function is called.

In the receiving thread, the VCI_Receive function is generally called cyclically to query the receiving buffer and receive data. The key parameters are: call frequency, receive length Len. Generally Len is set to the length of the storage space, or according to the actual receive frequency and bus message frequency to set the appropriate value, provided that the receive array should be large enough, otherwise the phenomenon of frame loss may occur.

Call frequency: call VCI_Receive function, essentially read and write USB. When reading and writing USB has a minimum time-consuming, which is generally 0.15ms. Therefore, VCI_Receive function call interval should be set to at least 0.15ms or more.

Call frequency, receive length Len two parameters directly affect the performance of the adapter. Assuming that the bus rate is 10000 frames/s and the receive length Len is 2500 frames, the function should be called at least 4 times per second, i.e., the calling interval is less than 250 ms. If the receive length Len is set to 1000 frames, the function should be called at least 10 times per second, i.e., the calling interval is less than 100 ms; otherwise, the receive buffer will be overflowed. In this way, we can match the call frequency and receive length Len to get the optimal value.

7、VCI_Transmit Function

This function must be called after the CAN channel is activated, and it is used to send buffer to the CAN channel (USB_CAN adapter provides about 10 frames of CAN message length per channel to send buffer, and each time VCI_Transmit is called, it sends about 10 frames of data at most.) The data is sent and then sent out through the CAN bus. Its formal parameter pSend is the first pointer of the array of data frames to be sent; the other formal parameter len is the number of data frames to be sent, which can be set to any integer between 1 and 10.

The sending speed of the sending device is determined by the performance of the current computer hardware and software, and the general continuous sending speed is

more than 8000 fps (1Mbps). If the sending speed is too fast, it will be possible to make the bus utilization reach 100%, so that the sender will have a sending timeout. The user can add appropriate delay in the application program to reduce the speed of sending.

Each call to the VCI_Transmit function in the sending process has a timeout limit, and the timeout time when sending is about 1 s. The sending timeout generally occurs when the CAN bus is busy and the current node has a low priority, and it is not a function call or a communication error, which can be programmed by the user to realize the retransmission. Therefore, care should be taken to ensure that the CAN bus occupancy should not exceed 60-70% of the bus capacity during system design.

Call frequency: call VCI_Transmit function, which essentially is read and write USB. Reading and writing USB has a minimum time-consuming, generally 0.15ms, VCI_Transmit function call interval should be set to at least 0.15ms above.

The number of frames sent each time is recommended to be set to 1, a single frame is sent each time. Each send interval can not be too small, otherwise the bus occupancy rate of 100%, send buffer full. Due to the blocking mechanism, call VCI_Transmit function will timeout to return to the call VCI_Transmit function will not return immediately, but wait for 1s until the timeout to return to the application is to increase the occupied resources, and the interface response is sluggish. So the parameter setting is important.

When the USB-CAN adapter plays a role similar to a master, for example, sending query commands to each slave node of the bus and receiving data returned from each node, each call to the VCI_Transmit function can only be sent in a single frame, and the interval between the sends should be calculated to avoid the instantaneous load rate of the bus is too high, which leads to the shutdown of USBCAN as the transmitting node. If more than one frame is sent at a time, the data has not yet been sent. At this time the slave nodes above the bus have started to receive the query command and return data. The slave nodes sending data and the USBCAN as the master sending data will occur at the same time, and the bus will enter a busy state. If the USBCAN sends too many frames at once, the transmit error

counter will accumulate. If it accumulates too quickly and reaches a certain value, the USBCAN will enter the shutdown state. In the off state, the unsent data will wait to be sent and will not be lost, and the USBCAN will not receive the data on the bus. Therefore, the query command will be sent to all nodes, but some of the returned data will be lost during the USBCAN shutdown period, resulting in incomplete reception by the USBCAN. After the USBCAN is shut down and the bus is idle, it will automatically revert to the normal operating state, and continue to send and receive. This situation is determined by the error handling mechanism of the CAN bus node, as long as the CAN node (USBCAN as master is also a CAN node, slave is also a CAN node) will have such an error handling mechanism to avoid the bus being occupied by a single node. So you should calculate the number of nodes, the number of query commands and return data, set the load ratio, and distribute the bus evenly.to fix the sending interval.

8、 VCI_FindUsbDevice and VCI_FindUsbDevice2 function

This function can be called at any time to query the computer all the information inserted into the USB-CAN adapter, and return to the first four adapter information. If the computer is inserted into more than four adapters, then use the VCI_FindUsbDevice2 function, the maximum support for 50 USB-CAN adapters.